

Lean Formalization Plan for *A Self-Contained Proof of Sharpness for Nearest-Neighbor Bond Percolation on $\mathbb{Z}^d$*

Implementation blueprint for Lean 4 + mathlib + Codex

June 2026

Abstract

This document gives a modular formalization plan for the sharpness proof for nearest-neighbor Bernoulli bond percolation on $\mathbb{Z}^d$. The plan is designed for a Codex-driven Lean implementation. The central strategy is to avoid building the infinite product measure at the beginning. Instead, all probabilities are finite Bernoulli product probabilities attached to finite edge supports, and the infinite-volume quantity $\theta(p)$ is defined as the monotone limit of finite exit probabilities. The proof is split into geometry, connectivity, finite product probability, independence, Russo’s formula, the finite-set criterion, the differential inequality, the supercritical bound, the boundary inequality, subcritical exponential decay, and critical-point identification.

Contents

1	Target theorem and formalization strategy	1
1.1	Main Lean design choice: finite-volume probability first	1
1.2	Auxiliary finite-set critical point	2
2	Recommended Lean file structure	2
3	Fact graph	3
4	Part I: $\mathbb{Z}^d$ geometry and bonds	4
4.1	Vertices, norm, adjacency, and boxes	4
4.2	Unoriented bonds and oriented boundaries	5
5	Part II: paths, connectivity, and clusters	6
5.1	Paths	6
5.2	Finite exit event	6
5.3	Clusters inside a finite set	6
6	Part III: finite Bernoulli product probability	7
6.1	Finite product probability by sums	7
6.2	Local events and support extension	7
6.3	Independence API	8
6.4	Monotonicity in p	8

7	Part IV: Russo’s formula	8
7.1	Closed-pivotal form	8
7.2	Lean proof plan for Russo	9
8	Part V: the finite-set criterion $\varphi_p(S)$ and $\tilde{p}_c$	9
8.1	Definition of $\varphi_p(S)$	9
8.2	Order consequences	9
9	Part VI: Lemma 2, the fundamental differential inequality	10
9.1	Statement	10
9.2	The random shield set	10
9.3	Pivotal equivalence	10
9.4	Differential inequality proof chain	11
10	Part VII: Lemma 3, supercritical lower bound	11
10.1	Generic calculus lemma	11
10.2	Supercritical proof chain	12
11	Part VIII: Lemma 4, the boundary inequality	12
11.1	Finite-volume version recommended for Lean	12
11.2	Proof chain	12
12	Part IX: Lemma 5, subcritical exponential decay	13
12.1	Small n and endpoint $p = 0$	14
13	Part X: θ, p_c, and identification of critical points	14
13.1	Defining θ	14
13.2	Critical point	14
14	Implementation milestones for Codex	15
15	Risk register and recommended simplifications	15
16	Minimal theorem dependency table	16
17	Suggested Codex task prompts	17
18	Final checklist before attempting the main theorem	18

1 Target theorem and formalization strategy

The mathematical target is the following sharpness statement. Let $d \geq 2$, let vertices be $\mathbb{Z}^d$, and let nearest-neighbor unoriented bonds be open independently with probability $p \in [0, 1]$. Let

$$\Lambda_n := \{x \in \mathbb{Z}^d : \|x\|_1 \leq n\}, \quad a_n(p) := \mathbb{P}_p(0 \longleftrightarrow \Lambda_n^c).$$

Define

$$\theta(p) := \lim_{n \rightarrow \infty} a_n(p), \quad p_c := \inf\{p \in [0, 1] : \theta(p) > 0\}.$$

The final theorem is:

- (i) if $p < p_c$, then $a_n(p) \leq e^{-cn}$ for some $c = c(p) > 0$ and every n ;
- (ii) if $p_c < p < 1$, then

$$\theta(p) \geq \frac{p - p_c}{p(1 - p_c)}.$$

1.1 Main Lean design choice: finite-volume probability first

A direct infinite-product construction is unnecessary for this proof and would make the Lean project much harder. Instead:

- define a global configuration type $\Omega_d := \text{Bond}_d \rightarrow \text{Bool}$ only as a semantic object;
- define a *local event* as a predicate on Ω_d together with a finite edge support on which it depends;
- define $\mathbb{P}_p(E)$ for a local event by a finite sum over assignments on its finite support;
- define $a_n(p)$ as the finite probability of the local event “0 exits Λ_n ”;
- define $\theta(p)$ as $\inf_n a_n(p)$, or equivalently as the limit of the decreasing sequence $a_n(p)$.

This replacement turns all probabilistic statements in the proof into finite combinatorics plus real analysis. The only limiting argument needed is monotone convergence of a decreasing sequence of real numbers in $[0, 1]$, not measure-theoretic continuity from above for an infinite product measure.

1.2 Auxiliary finite-set critical point

For a finite set $S \subset \mathbb{Z}^d$ with $0 \in S$, define the oriented edge boundary

$$\partial_E S := \{(x, y) : x \in S, y \notin S, \|x - y\|_1 = 1\},$$

and

$$\varphi_p(S) := p \sum_{(x,y) \in \partial_E S} \mathbb{P}_p(0 \xrightarrow{S} x).$$

Define

$$\tilde{p}_c := \sup\{p \in [0, 1] : \exists \text{ finite } S \ni 0 \text{ with } \varphi_p(S) < 1\}.$$

The implementation proves the theorem first with $\tilde{p}_c$ in place of p_c , and then proves $\tilde{p}_c = p_c$.

2 Recommended Lean file structure

The project should be built from small files. Each file should expose theorem statements that later files consume. Codex should never be asked to prove a late theorem before all earlier interfaces compile.

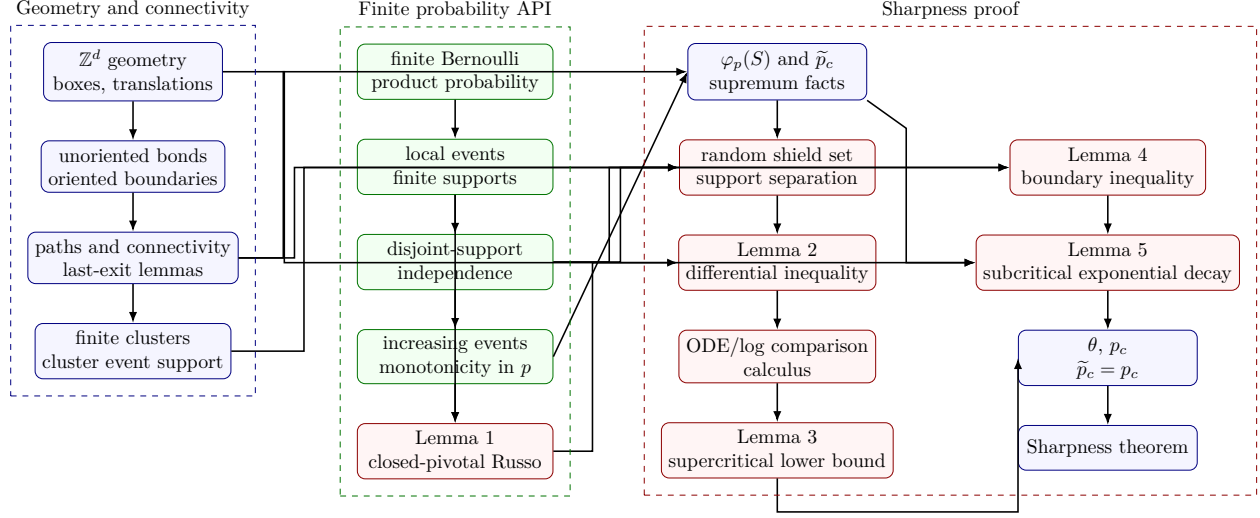
Lean file	Contents	Depends on
Zd	Vertices $\mathbb{Z}^d$, ℓ^1 norm, adjacency, translation, boxes Λ_n .	mathlib finite sums, integers.
Bonds	Unoriented nearest-neighbor bonds, canonical bond from adjacent vertices, oriented boundary edges.	Zd.

Lean file	Contents	Depends on
FiniteGeometry	Finite internal edge sets, exit edge supports, boundary endpoint bounds, ℓ^1 triangle arguments.	Zd, Bonds.
Paths	Lists as paths, path concatenation, first/last exit lemmas, connectivity inside a finite set.	Bonds, FiniteGeometry.
Clusters	Finite cluster $C_S(u)$, cluster event $C_S(u) = C_0$, cluster-boundary facts.	Paths.
FiniteBernoulli	Finite Bernoulli product probability by finite sums, total mass, complement, union bound.	mathlib algebra and finite sums.
LocalEvent	Local events, finite supports, support extension invariance, event operations.	FiniteBernoulli, Bonds.
Independence	Disjoint-support multiplication and finite independence for two and three local events.	LocalEvent.
Monotonicity	Increasing events, monotonicity of $p \mapsto \mathbb{P}_p(E)$, monotonicity of $\varphi_p(S)$.	LocalEvent.
Russo	Closed-pivotal Russo formula for finite local increasing events.	LocalEvent, Independence, calculus.
Events	Local events $x \xleftrightarrow{S} y$, $x \longleftrightarrow \Lambda_n^c$, finite exit event A_Λ .	Paths, LocalEvent.
Phi	$\varphi_p(S)$, $\tilde{p}_c$, elementary consequences of the supremum definition.	Events, Monotonicity.
DiffIneq	Lemma 2: fundamental differential inequality.	Russo, Phi, Events.
OdeComparison	Generic calculus lemma integrating $f' \geq (1-f)/(p(1-p))$.	real calculus.
Supercritical	Lemma 3: lower bound above $\tilde{p}_c$.	DifferentialInequality, OdeComparison.
BoundaryIneq	Lemma 4: finite-set expansion inequality.	Clusters, Independence.
Subcritical	Lemma 5: exponential decay below $\tilde{p}_c$.	BoundaryInequality, Phi, FiniteGeometry.
CriticalPoint	Definition of θ , p_c , proof $\tilde{p}_c = p_c$.	Supercritical, Subcritical.
Main	Final theorem with p_c .	CriticalPoint.

3 Fact graph

The following graph is the dependency skeleton. A solid arrow means a theorem directly uses the previous block. The large boxes show the three foundations: geometry/connectivity, finite

probability, and real-order/calculus.



4 Part I: $\mathbb{Z}^d$ geometry and bonds

4.1 Vertices, norm, adjacency, and boxes

Use

$$\text{Vertex}(d) := \text{Fin}(d) \rightarrow \mathbb{Z}.$$

Define

$$\|x\|_1 := \sum_{i \in \text{Fin}(d)} |x_i|, \quad x \sim y \iff \|x - y\|_1 = 1.$$

Define Λ_n as a **Finset** by filtering the finite product box $[-n, n]^d$ by the condition $\|x\|_1 \leq n$. Avoid using an infinite set plus a separate finiteness proof whenever possible.

Lean interface 4.1 (core geometry statements). The following are the Lean-level facts needed later.

```

def Vertex (d : Nat) := Fin d -> Int

def l1 (x : Vertex d) : Nat := ...
def Adj (x y : Vertex d) : Prop := l1 (x - y) = 1

def ball (d : Nat) (n : Nat) : Finset (Vertex d) := ...

lemma mem_ball_iff : x in ball d n <=> l1 x <= n
lemma zero_mem_ball : (0 : Vertex d) in ball d n
lemma adj_l1_triangle : Adj x y -> l1 z <= n -> ...
lemma translate_ball_exit :
  y in ball d L -> z notin ball d n -> z - y notin ball d (n - L)

def translateVertex (a x : Vertex d) : Vertex d := x + a
lemma adj_translate_iff : Adj (x+a) (y+a) <=> Adj x y
  
```

The key geometric lemma for subcritical decay is:

Lemma 4.2 (translation-to-origin estimate). *If $y \in \Lambda_L$ and $z \notin \Lambda_n$, then $z - y \notin \Lambda_{n-L}$ for $n \geq L$. Equivalently, any path from y to Λ_n^c translates to a path from 0 to Λ_{n-L}^c .*

Proof idea. Use the ℓ^1 triangle inequality:

$$\|z - y\|_1 \geq \|z\|_1 - \|y\|_1 > n - L.$$

In Lean it is often easier to prove the contrapositive using natural-valued inequalities: $\|z - y\|_1 \leq n - L$ and $\|y\|_1 \leq L$ imply $\|z\|_1 \leq n$. $\square$

4.2 Unoriented bonds and oriented boundaries

The percolation state should live on unoriented bonds. Boundary sums, however, are naturally over oriented pairs (x, y) with $x \in S$, $y \notin S$. Recommended design:

- define an unoriented bond as a two-element finite set of adjacent vertices, or as a canonical sorted pair;
- define `bondOfAdj x y` for adjacent vertices;
- define oriented boundary edges as vertex pairs, with a projection to the associated unoriented bond.

Lean interface 4.3 (bonds and boundaries).

```
structure Bond (d : Nat) where
  carrier : Finset (Vertex d)
  card_two : carrier.card = 2
  adj_pair : forall x in carrier, forall y in carrier,
    x != y -> Adj x y

def bondOfAdj (hxy : Adj x y) : Bond d := ...

structure OEdgeBoundary (S : Finset (Vertex d)) where
  x : Vertex d
  y : Vertex d
  hx : x in S
  hy : y notin S
  hxy : Adj x y

def OEdgeBoundary.bond (e : OEdgeBoundary S) : Bond d := bondOfAdj e.hxy

def internalBonds (S : Finset (Vertex d)) : Finset (Bond d) := ...
def exitBonds (S : Finset (Vertex d)) : Finset (Bond d) := ...
```

The most important support-disjointness facts are:

$$\begin{aligned} \text{Int}(S) &:= \{\{x, y\} : x, y \in S, x \sim y\}, \\ \text{Ext}(S) &:= \text{finite ambient support} \setminus \text{Int}(S), \\ \text{Int}(S) \cap \text{Ext}(S) &= \emptyset. \end{aligned}$$

For Lemma 4, also define

$$E(S, C_0) := \{\{a, b\} : a, b \in S, \{a, b\} \cap C_0 \neq \emptyset\}.$$

Then $E(S, C_0)$ is disjoint from a boundary edge xy with $x \in C_0$, $y \notin S$, and from the edges used by paths in $\mathbb{Z}^d \setminus C_0$.

5 Part II: paths, connectivity, and clusters

5.1 Paths

Represent paths by lists of vertices. A path is valid if consecutive vertices are adjacent. It is open in a configuration if every consecutive bond is open. It stays in S if every vertex of the list belongs to S .

Lean interface 5.1 (path and connectivity definitions).

```
def IsPath (gamma : List (Vertex d)) : Prop := ...
def PathFromTo (gamma : List (Vertex d)) (x y : Vertex d) : Prop := ...
def PathIn (S : Finset (Vertex d)) (gamma : List (Vertex d)) : Prop := ...
def OpenPath (omega : Config d) (gamma : List (Vertex d)) : Prop := ...

def ConnIn (omega : Config d) (S : Finset (Vertex d))
  (x y : Vertex d) : Prop :=
  exists gamma, PathFromTo gamma x y and PathIn S gamma and OpenPath omega gamma
```

Do not initially try to build a sophisticated graph library. The proof only needs path reversal, path concatenation, translation of paths, and last-exit facts for finite lists.

5.2 Finite exit event

For a finite $\Lambda \ni 0$, define the finite exit event A_Λ by first reducing the informal event $\{0 \longleftrightarrow \Lambda^c\}$ to a boundary-crossing event. Namely,

$$A_\Lambda(\omega) \iff \exists(x, y) \in \partial_E \Lambda, 0 \xleftrightarrow{\Lambda} x \text{ and } \omega(\{x, y\}) = \text{open}.$$

Thus A_Λ is a local event supported on internal bonds of Λ plus boundary bonds of Λ .

Lean interface 5.2 (finite exit event).

```
def ExitEvent (Lam : Finset (Vertex d)) (h0 : 0 in Lam) : LocalEvent d := ...
def exitProb (p : Real) (Lam : Finset (Vertex d)) : Real :=
  Prob p (ExitEvent Lam h0)

def boxExitProb (p : Real) (n : Nat) : Real :=
  exitProb p (ball d n)
```

This definition replaces the infinite event $\{0 \longleftrightarrow \Lambda^c\}$ everywhere. It is mathematically equivalent for nearest-neighbor finite Λ because an exit path has a first edge leaving Λ .

5.3 Clusters inside a finite set

For Lemma 4 define

$$C_S(u; \omega) := \{x \in S : u \xleftrightarrow{S} x\}.$$

The event $C_S(u) = C_0$ is local. Its minimal support is not all of $\text{Int}(S)$; it suffices to use the bonds inside S with at least one endpoint in C_0 . This sharper support statement is useful for independence.

Lemma 5.3 (cluster event support). *On the event $C_S(u) = C_0$, whether the event holds is determined by the states of bonds $\{a, b\}$ with $a, b \in S$ and at least one endpoint in C_0 .*

Proof idea. To know $C_S(u) = C_0$, it is enough to know that all vertices of C_0 are connected to u inside C_0 using open bonds, and all boundary bonds from C_0 to $S \setminus C_0$ are closed. Edges wholly inside $S \setminus C_0$ cannot change the cluster of u . $\square$

6 Part III: finite Bernoulli product probability

6.1 Finite product probability by sums

For a finite edge set F , let $\{0, 1\}^F$ be assignments of open/closed states. Define

$$w_p(\sigma) := \prod_{e \in F} \begin{cases} p, & \sigma(e) = 1, \\ 1 - p, & \sigma(e) = 0, \end{cases}$$

and

$$\text{Prob}_p^F(A) := \sum_{\sigma \in \{0,1\}^F} \mathbf{1}_A(\sigma) w_p(\sigma).$$

Use real-valued probabilities, not `ENNReal`, because Russo's formula and the supercritical integration are real-calculus statements.

Lean interface 6.1 (finite Bernoulli probability).

```
def weight (p : Real) (F : Finset (Bond d)) (sigma : F -> Bool) : Real := ...
def ProbOn (p : Real) (F : Finset (Bond d)) (A : (F -> Bool) -> Prop) : Real :=
  sum sigma, if A sigma then weight p F sigma else 0

lemma prob_nonneg (hp0 : 0 <= p) (hp1 : p <= 1) : 0 <= ProbOn p F A
lemma prob_le_one (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F A <= 1
lemma prob_univ (hp0 : 0 <= p) (hp1 : p <= 1) : ProbOn p F True = 1
lemma prob_compl : ProbOn p F (not A) = 1 - ProbOn p F A
lemma prob_union_bound : ProbOn p F (A or B) <= ProbOn p F A + ProbOn p F B
```

6.2 Local events and support extension

To avoid changing probability spaces constantly, wrap local predicates as predicates on global configurations plus a finite support.

Lean interface 6.2 (local events).

```
abbrev Config (d : Nat) := Bond d -> Bool

def DependsOn (F : Finset (Bond d)) (A : Config d -> Prop) : Prop :=
  forall omega omega', (forall e in F, omega e = omega' e) -> (A omega <-> A omega')

structure LocalEvent (d : Nat) where
  support : Finset (Bond d)
  pred : Config d -> Prop
  local : DependsOn support pred

def Prob (p : Real) (E : LocalEvent d) : Real := ...
```

Essential support lemmas:

- (S1) If E depends on F and $F \subset G$, then computing $\mathbb{P}_p(E)$ using F or G gives the same number.
- (S2) $E \cap E'$, $E \cup E'$, and E^c are local, with support contained in the union of supports.
- (S3) If E and E' have disjoint supports, then $\mathbb{P}_p(E \cap E') = \mathbb{P}_p(E)\mathbb{P}_p(E')$.
- (S4) The same multiplication lemma should be available for three events with pairwise disjoint supports.

6.3 Independence API

The independence lemmas are used in only two places: Lemma 2 and Lemma 4. A specialized disjoint-support theorem is enough.

Lemma 6.3 (disjoint-support independence). *Let E_1, E_2 be local events with disjoint supports. For $0 \leq p \leq 1$,*

$$\mathbb{P}_p(E_1 \cap E_2) = \mathbb{P}_p(E_1)\mathbb{P}_p(E_2).$$

If E_1, E_2, E_3 have pairwise disjoint supports, then

$$\mathbb{P}_p(E_1 \cap E_2 \cap E_3) = \mathbb{P}_p(E_1)\mathbb{P}_p(E_2)\mathbb{P}_p(E_3).$$

Proof idea. Reduce to finite products over $F_1 \cup F_2$ or $F_1 \cup F_2 \cup F_3$ and use factorization of finite sums over product types. This is purely finite algebra. $\square$

6.4 Monotonicity in p

The definition of $\tilde{p}_c$ uses monotonicity of $\varphi_p(S)$. Prove monotonicity of probabilities of increasing local events once.

Definition 6.4 (increasing local event). A local event E is increasing if $\omega \leq \omega'$ coordinatewise and $E(\omega)$ imply $E(\omega')$.

Lemma 6.5 (Bernoulli monotonicity). *If E is increasing and $0 \leq p \leq q \leq 1$, then*

$$\mathbb{P}_p(E) \leq \mathbb{P}_q(E).$$

Proof idea. Use the standard coupling: give each edge an independent uniform variable in a finite ordered grid, or prove directly by induction on the finite support. The induction proof is usually easier in Lean: condition on one edge and use

$$\mathbb{P}_p(E) = (1 - p)\mathbb{P}_p(E|e = 0) + p\mathbb{P}_p(E|e = 1),$$

with $\mathbb{P}_p(E|e = 0) \leq \mathbb{P}_p(E|e = 1)$ for increasing E . $\square$

Then $p \mapsto \varphi_p(S)$ is nondecreasing because it is a finite sum of products of p and increasing connection probabilities.

7 Part IV: Russo's formula

7.1 Closed-pivotal form

For a local increasing event A and an edge e in its support, define “ e is closed-pivotal” as:

- e is closed;
- A does not occur;
- after forcing e to be open, A occurs.

Theorem 7.1 (Lemma 1, Russo closed-pivotal form). *If A is increasing and local, then for $0 < p < 1$,*

$$\frac{d}{dp}\mathbb{P}_p(A) = \frac{1}{1-p} \sum_{e \in \text{supp}(A)} \mathbb{P}_p(e \text{ is closed-pivotal for } A).$$

7.2 Lean proof plan for Russo

Implement Russo in a generic finite Boolean product file, independent of percolation.

1. For an event $A \subseteq \{0, 1\}^F$, define the edge-influence event B_e on $F \setminus \{e\}$: changing e changes A .
2. Prove the single-coordinate derivative identity

$$\frac{\partial}{\partial p_e} \mathbb{P}_{(p_f)}(A) = \mathbb{P}_{(p_f)_{f \neq e}}(B_e).$$

3. Specialize to the homogeneous case $p_f = p$ and sum over $e \in F$.
4. For increasing A , prove

$$\mathbb{P}_p(e \text{ closed-pivotal}) = (1 - p) \mathbb{P}_p(B_e).$$

The most Codex-friendly route is an induction on the finite list of edges, not a high-level measure-theory proof. Keep all differentiability statements in $\mathbb{R}$.

Lean interface 7.2 (Russo theorem stub).

```
theorem russo_closed_pivotal
  (E : LocalEvent d) (hinc : Increasing E)
  {p : Real} (hp0 : 0 < p) (hp1 : p < 1) :
  HasDerivAt (fun q => Prob q E)
    ((1 / (1 - p)) * sum e in E.support, Prob p (closedPivotalEvent E e)) p := ...
```

8 Part V: the finite-set criterion $\varphi_p(S)$ and $\tilde{p}_c$

8.1 Definition of $\varphi_p(S)$

For finite $S \ni 0$,

$$\varphi_p(S) = p \sum_{(x,y) \in \partial_E S} \mathbb{P}_p(0 \xleftrightarrow{S} x).$$

The event $0 \xleftrightarrow{S} x$ is local with support $\text{Int}(S)$.

Lean interface 8.1 (phi and p-tilde).

```
def phi (p : Real) (S : Finset (Vertex d)) : Real :=
  p * sum e in orientedBoundary S, Prob p (connInEvent S 0 e.x)

def pTilde : Real :=
  sSup {p : Real | 0 <= p and p <= 1 and exists S, finite S and 0 in S and phi p S < 1}
```

8.2 Order consequences

Prove the two consequences as standalone order lemmas.

Lemma 8.2 (below $\tilde{p}_c$). *If $p < \tilde{p}_c$, then there exists a finite $S \ni 0$ such that $\varphi_p(S) < 1$.*

Proof idea. By the definition of supremum, choose $q > p$ in the set. Then choose S with $\varphi_q(S) < 1$. Monotonicity gives $\varphi_p(S) \leq \varphi_q(S) < 1$. $\square$

Lemma 8.3 (above $\tilde{p}_c$). *If $p > \tilde{p}_c$, then for every finite $S \ni 0$, $\varphi_p(S) \geq 1$.*

Proof idea. Otherwise p itself belongs to the set defining $\tilde{p}_c$, contradicting $p > \sup$. $\square$

These two lemmas are repeatedly used and should not be reproved inside later files.

9 Part VI: Lemma 2, the fundamental differential inequality

9.1 Statement

For finite $\Lambda \ni 0$, define $f_\Lambda(p) := \mathbb{P}_p(A_\Lambda)$. Lemma 2 states:

$$f'_\Lambda(p) \geq \frac{1}{p(1-p)} \inf_{\substack{S \subset \Lambda \\ 0 \in S}} \varphi_p(S) (1 - f_\Lambda(p)).$$

In Lean, since the infimum is over finitely many subsets of Λ , use a finite minimum rather than a set-theoretic infimum.

Lean interface 9.1 (finite minimum version).

```
def phiMinIn (p : Real) (Lam : Finset (Vertex d)) : Real :=
  Finset.inf' {S : Finset (Vertex d) | S subset Lam and 0 in S} ... (phi p S)

theorem differential_inequality
  (Lam : Finset (Vertex d)) (h0 : 0 in Lam)
  {p : Real} (hp0 : 0 < p) (hp1 : p < 1) :
  deriv (fun q => exitProb q Lam) p >=
    (1 / (p * (1 - p))) * phiMinIn p Lam * (1 - exitProb p Lam) := ...
```

9.2 The random shield set

Define

$$\mathcal{S}(\omega) := \{x \in \Lambda : x \not\longleftrightarrow \Lambda^c\}.$$

In the finite formalization, $x \longleftrightarrow \Lambda^c$ means the finite exit event from x through $\partial_E \Lambda$. Then

$$0 \in \mathcal{S}(\omega) \iff A_\Lambda(\omega)^c.$$

Lemma 9.2 (support separation for $\{\mathcal{S} = S\}$). *Fix $S \subset \Lambda$. The event $\{\mathcal{S} = S\}$ is determined by the edge states not internal to S .*

Proof idea. Delete all internal edges of S . Then $\{\mathcal{S} = S\}$ is equivalent to:

- (a) no vertex of S is connected to Λ^c in the deleted-internal-edge graph;
- (b) every vertex of $\Lambda \setminus S$ is connected to Λ^c in the deleted-internal-edge graph.

Both conditions use only non-internal edges. The nontrivial direction is a last-exit argument: if a path using internal S -edges connects a vertex of S to Λ^c , take the last exit from S ; the tail witnesses a connection to Λ^c without internal S -edges. The same argument handles vertices of $\Lambda \setminus S$. $\square$

9.3 Pivotal equivalence

Fix $S \subset \Lambda$ with $0 \in S$ and an oriented boundary edge $(x, y) \in \partial_E S$. On $\{\mathcal{S} = S\}$,

$$\{xy \text{ is closed-pivotal for } A_\Lambda\} \iff \{0 \xleftrightarrow{S} x\}.$$

The edge xy is automatically closed on the right-hand side; otherwise x would connect to Λ^c , contradicting $x \in \mathcal{S}$.

9.4 Differential inequality proof chain

The formal proof should follow this exact chain.

1. Apply Russo to A_Λ :

$$f'_\Lambda(p) = \frac{1}{1-p} \sum_e \mathbb{P}_p(e \text{ closed-pivotal}).$$

2. Decompose by values of $\mathcal{S}$ with $0 \in S$.
3. Restrict the sum to closed-pivotal edges oriented from S to S^c .
4. Use pivotal equivalence:

$$\mathbb{P}_p(e \text{ closed-pivotal}, \mathcal{S} = S) = \mathbb{P}_p(0 \xrightarrow{S} x, \mathcal{S} = S).$$

5. Use support separation and disjoint-support independence:

$$\mathbb{P}_p(0 \xrightarrow{S} x, \mathcal{S} = S) = \mathbb{P}_p(0 \xrightarrow{S} x) \mathbb{P}_p(\mathcal{S} = S).$$

6. Sum over $(x, y) \in \partial_E S$ and use the definition of $\varphi_p(S)$:

$$\sum_{(x,y) \in \partial_E S} \mathbb{P}_p(0 \xrightarrow{S} x) \mathbb{P}_p(\mathcal{S} = S) = \frac{\varphi_p(S)}{p} \mathbb{P}_p(\mathcal{S} = S).$$

7. Bound $\varphi_p(S)$ below by the finite minimum over $S \subset \Lambda$, $0 \in S$.
8. Sum $\mathbb{P}_p(\mathcal{S} = S)$ over all $S \subset \Lambda$ with $0 \in S$, obtaining

$$\mathbb{P}_p(0 \in \mathcal{S}) = 1 - f_\Lambda(p).$$

This is one of the two hardest mathematical modules. The recommended Codex workflow is: first formalize the statement with the support-separation and pivotal-equivalence lemmas as named hypotheses; then prove those two lemmas separately.

10 Part VII: Lemma 3, supercritical lower bound

10.1 Generic calculus lemma

Isolate the real-analysis step.

Lemma 10.1 (ODE/log comparison). *Let $0 < q < p < 1$. Suppose $f : [q, p] \rightarrow \mathbb{R}$ is differentiable, $f(t) < 1$, and*

$$f'(t) \geq \frac{1 - f(t)}{t(1 - t)} \quad (q < t < p).$$

Then

$$1 - f(p) \leq (1 - f(q)) \frac{q(1 - p)}{p(1 - q)}.$$

Proof idea. Apply the derivative identity

$$\frac{d}{dt} \log(1 - f(t)) = -\frac{f'(t)}{1 - f(t)}$$

and integrate:

$$\log \frac{1 - f(p)}{1 - f(q)} \leq -\int_q^p \frac{dt}{t(1 - t)} = -\log \frac{p(1 - q)}{q(1 - p)}.$$

The side condition $f(t) < 1$ follows for finite exit events by closing all boundary edges of Λ . $\square$

10.2 Supercritical proof chain

For $\tilde{p}_c < p < 1$:

1. Fix finite $\Lambda \ni 0$.
2. For every $q > \tilde{p}_c$, the order lemma gives $\varphi_q(S) \geq 1$ for all finite $S \ni 0$.
3. Lemma 2 gives

$$f'_\Lambda(q) \geq \frac{1}{q(1-q)}(1 - f_\Lambda(q)).$$

4. Apply ODE/log comparison from q to p .
5. Use $1 - f_\Lambda(q) \leq 1$:

$$f_\Lambda(p) \geq 1 - \frac{q(1-p)}{p(1-q)}.$$

6. Let $q \downarrow \tilde{p}_c$ using elementary real continuity of the rational expression.
7. Take $\Lambda = \Lambda_n$ and then $n \rightarrow \infty$ using the definition of θ as the decreasing limit of $a_n(p)$.

Therefore

$$\theta(p) \geq \frac{p - \tilde{p}_c}{p(1 - \tilde{p}_c)}.$$

11 Part VIII: Lemma 4, the boundary inequality

11.1 Finite-volume version recommended for Lean

The article states a general inequality with $B \subset \mathbb{Z}^d$, $B \cap S = \emptyset$. For the Lean proof, formalize a finite-volume version first. Let T be a finite ambient vertex set containing S and B , and let $u \in S$, $B \cap S = \emptyset$. Then

$$\mathbb{P}_p(u \xleftrightarrow{T} B) \leq \sum_{(x,y) \in \partial_E S} p \mathbb{P}_p(u \xleftrightarrow{S} x) \mathbb{P}_p(y \xleftrightarrow{T} B).$$

The only application needed for subcritical decay is $u = 0$ and $B = \Lambda_n^c$ in the finite exit-event sense.

11.2 Proof chain

Let

$$C = C_S(u) = \{x \in S : u \xleftrightarrow{S} x\}.$$

For a possible value $C_0 \subset S$ with $u \in C_0$:

1. On $\{C = C_0\}$ and $\{u \longleftrightarrow B\}$, choose an open path from u to B .
2. Take the last vertex x of the path lying in C_0 and the next vertex y .
3. Then $x \in C_0$, $y \notin S$, $x \sim y$, the edge xy is open, and the remaining tail connects y to B while avoiding C_0 .
4. Hence

$$\{u \longleftrightarrow B, C = C_0\} \subseteq \bigcup_{x \in C_0, y \notin S, x \sim y} \{C = C_0\} \cap \{xy \text{ open}\} \cap \{y \xleftrightarrow{\mathbb{Z}^d \setminus C_0} B\}.$$

5. Use the union bound.

6. The three events have disjoint supports:

- $\{C = C_0\}$ uses only internal S -edges incident to C_0 ;
- $\{xy \text{ open}\}$ uses the single boundary bond xy ;
- $\{y \xleftrightarrow{\mathbb{Z}^d \setminus C_0} B\}$ uses no edge incident to C_0 .

7. Apply independence and then drop the avoidance condition:

$$\mathbb{P}_p(y \xleftrightarrow{\mathbb{Z}^d \setminus C_0} B) \leq \mathbb{P}_p(y \longleftrightarrow B).$$

8. Sum over C_0 and identify

$$\sum_{C_0 \ni x} \mathbb{P}_p(C = C_0) = \mathbb{P}_p(x \in C_S(u)) = \mathbb{P}_p(u \xleftrightarrow{S} x).$$

Lean interface 11.1 (boundary inequality theorem stub). □

`theorem boundary_inequality`

```
(S : Finset (Vertex d)) (u : Vertex d) (hu : u in S)
(B : Finset (Vertex d)) (hBS : Disjoint B S)
{p : Real} (hp0 : 0 <= p) (hp1 : p <= 1) :
Prob p (connToSetEvent u B) <=
  sum e in orientedBoundary S,
    p * Prob p (connInEvent S u e.x) * Prob p (connToSetEvent e.y B) := ...
```

If the fully general B version causes too much overhead, prove the specialized version with $B = \Lambda_n^c$ as an exit event. This is sufficient for the final theorem.

12 Part IX: Lemma 5, subcritical exponential decay

Assume $p < \tilde{p}_c$. Choose finite $S \ni 0$ with

$$\rho := \varphi_p(S) < 1.$$

Choose $L \geq 1$ such that $S \subset \Lambda_{L-1}$. Then every boundary endpoint y in $\partial_E S$ lies in Λ_L .

Let

$$a_n := \mathbb{P}_p(0 \longleftrightarrow \Lambda_n^c).$$

For $n \geq L$, Lemma 4 gives

$$a_n \leq \sum_{(x,y) \in \partial_E S} p \mathbb{P}_p(0 \xleftrightarrow{S} x) \mathbb{P}_p(y \longleftrightarrow \Lambda_n^c).$$

The translation estimate gives

$$\mathbb{P}_p(y \longleftrightarrow \Lambda_n^c) \leq a_{n-L}$$

for every boundary endpoint $y \in \Lambda_L$. Hence

$$a_n \leq \rho a_{n-L}.$$

Iterating gives

$$a_n \leq \rho^{\lfloor n/L \rfloor}.$$

The final conversion to e^{-cn} requires only real arithmetic and the fact $\rho < 1$.

Lean interface 12.1 (subcritical theorem stub).

```
theorem exponential_decay_below_pTilde
  {p : Real} (hp0 : 0 <= p) (hp : p < pTilde d) :
  exists c : Real, 0 < c and forall n : Nat,
    boxExitProb d p n <= Real.exp (-(c * n)) := ...
```

12.1 Small n and endpoint $p = 0$

Separate these cases explicitly.

- If $p = 0$, the event $0 \longleftrightarrow \Lambda_n^c$ is impossible for $n \geq 0$ except for harmless conventions at $n = 0$; the bound is immediate.
- For $0 < p < \tilde{p}_c$, $p < 1$ automatically. For the finitely many $1 \leq n < L$, prove $a_n < 1$ by closing all boundary bonds of Λ_n .
- Define

$$c := \min \left\{ \frac{-\log \rho}{2L}, \min_{1 \leq n < L} \frac{-\log a_n}{n} \right\} > 0.$$

13 Part X: θ , p_c , and identification of critical points

13.1 Defining θ

Since the sequence $a_n(p)$ is decreasing, define

$$\theta(p) := \inf_{n \in \mathbb{N}} a_n(p).$$

Then prove:

1. $0 \leq \theta(p) \leq 1$;
2. $\theta(p) \leq a_n(p)$ for all n ;
3. if $a_n(p) \rightarrow L$, then $L = \theta(p)$;
4. if $a_n(p) \leq e^{-cn}$ with $c > 0$, then $\theta(p) = 0$;
5. if $a_n(p) \geq b$ for all n , then $\theta(p) \geq b$.

13.2 Critical point

Define

$$p_c := \inf \{p \in [0, 1] : \theta(p) > 0\}.$$

Prove $\tilde{p}_c = p_c$ as follows.

First inequality: $p_c \leq \tilde{p}_c$. If $\tilde{p}_c < 1$, then for every $p \in (\tilde{p}_c, 1)$, Lemma 3 gives $\theta(p) > 0$, so $p_c \leq p$. Let $p \downarrow \tilde{p}_c$. If $\tilde{p}_c = 1$, then $p_c \leq 1 = \tilde{p}_c$ because $\theta(1) = 1$.

Second inequality: $\tilde{p}_c \leq p_c$. If $p < \tilde{p}_c$, Lemma 5 gives exponential decay and hence $\theta(p) = 0$. Thus no $p < \tilde{p}_c$ belongs to the set $\{p : \theta(p) > 0\}$, so $\tilde{p}_c \leq p_c$.

Lean interface 13.1 (critical-point identification).

```
def theta (p : Real) : Real := sInf {a : Real | exists n, a = boxExitProb d p n}
-- or, better, sInf (Set.range (boxExitProb d p))

def pCrit : Real := sInf {p : Real | 0 <= p and p <= 1 and 0 < theta d p}

theorem pTilde_eq_pCrit : pTilde d = pCrit d := ...
```

Be careful: if θ is defined as `sInf (range a)`, the set is nonempty automatically. Prove bounded-below by 0 before using order lemmas about `sInf`.

14 Implementation milestones for Codex

Milestone 14.1 (M0: project skeleton). Create the Lean project, all files above, and all theorem statements with `by sorry`. Run `lake build`. The final theorem statement must already match the mathematical target. From this point onward, Codex may fill proofs but must not weaken theorem statements.

Milestone 14.2 (M1: finite probability API). Prove `FiniteBernoulli`, `LocalEvent`, `Independence`, and `Monotonicity`. This milestone is independent of $\mathbb{Z}^d$. It should be tested on toy events involving two or three Boolean coordinates.

Milestone 14.3 (M2: geometry and connectivity). Prove $\mathbb{Z}^d$ boxes are finite, adjacency is translation invariant, exit events are local, and the translation-to-origin estimate. Then prove path last-exit lemmas and cluster support lemmas.

Milestone 14.4 (M3: Russo). Prove Lemma 1 in a generic finite Boolean product setting. This is a hard but isolated file. Once it compiles, do not modify its interface unless absolutely necessary.

Milestone 14.5 (M4: finite-set criterion and Lemma 2). Implement $\varphi_p(S)$, $\tilde{p}_c$, and the two order consequences of the definition. Then prove the random-shield support separation and pivotal equivalence. Finish Lemma 2.

Milestone 14.6 (M5: Lemmas 3 and 4). Prove the generic ODE/log comparison lemma, then supercritical lower bound. Separately prove boundary inequality using clusters and disjoint-support independence.

Milestone 14.7 (M6: Lemma 5 and main theorem). Use Lemma 4 plus translation invariance to prove exponential decay below $\tilde{p}_c$. Define θ and p_c , prove $\tilde{p}_c = p_c$, and close the final theorem.

15 Risk register and recommended simplifications

Risk	Why it is hard in Lean	Recommended simplification
Unoriented edge representation	Quotients or unordered pairs can create rewriting overhead.	Use a concrete bond structure with a canonical constructor from adjacent vertices. Keep boundary edges oriented separately.
Infinite product probability	Measure-theoretic construction and continuity from above would dominate the project.	Avoid it. Define θ as the decreasing limit of finite exit probabilities.
Russo formula	Differentiating finite products with dependent finite supports can be verbose.	Prove Russo for a generic finite Boolean product first, then wrap local events by support extension.
Lemma 2 support separation	The event $\{\mathcal{S} = S\}$ has a non-obvious support.	Prove a clean deleted-internal-edge characterization as an independent lemma.
Lemma 4 independence	The cluster event support must be sharper than all internal edges of S .	Use the support “edges inside S incident to C_0 ” and prove disjointness explicitly.
Calculus in Lemma 3	Interval integration and log derivative can be technical.	Isolate a generic ODE comparison theorem in one file. The percolation file should only call it.
Critical point order reasoning	Supremum/infimum edge cases, especially $\tilde{p}_c = 1$, are easy to miss.	Prove separate real-order lemmas for the two inequalities and handle endpoints explicitly.

16 Minimal theorem dependency table

Target result	Required previous results	Difficulty
Monotonicity of $\varphi_p(S)$	increasing connection event; finite Bernoulli monotonicity; finite sums preserve order.	medium
Lemma 1, Russo	finite product probability; derivative of finite product weights; closed-pivotal/influence equivalence.	hard
A_Λ local and increasing	finite exit-edge characterization; path/connectivity support; monotonicity of open paths.	medium
Shield support separation	deleted-internal-edge characterization; last-exit path lemma.	hard

Target result	Required previous results	Difficulty
Pivotal equivalence in Lemma 2	definition of $\mathcal{S}$; boundary orientation; connection inside S ; finite exit event.	hard
Lemma 2 differential inequality	Russo; pivotal equivalence; support separation; independence; finite sum decomposition by $\mathcal{S}$.	hard
Lemma 3 supercritical bound	Lemma 2; above- $\tilde{p}_c$ order lemma; $f_\Lambda < 1$ for $p < 1$; ODE/log comparison; limit defining θ .	hard
Lemma 4 boundary inequality	cluster definition; cluster support; last-exit lemma; three-event independence; union bound.	hard
Lemma 5 exponential decay	below- $\tilde{p}_c$ order lemma; Lemma 4; boundary endpoints in Λ_L ; translation-to-origin estimate; recurrence iteration.	medium-hard
$\tilde{p}_c = p_c$	Lemma 3; Lemma 5; definition of θ and p_c ; real-order infimum facts; endpoint $\tilde{p}_c = 1$.	medium
Final theorem	$\tilde{p}_c = p_c$; Lemma 3 rewritten with p_c ; Lemma 5 rewritten with p_c .	easy after dependencies

17 Suggested Codex task prompts

These are implementation prompts that keep Codex focused.

1. **Finite probability prompt.** “In `FiniteBernoulli.lean`, prove the finite-sum Bernoulli probability lemmas for a finite type of coordinates. Do not mention percolation or $\mathbb{Z}^d$. Use real-valued probabilities and assume $0 \leq p \leq 1$.”
2. **Support prompt.** “In `LocalEvent.lean`, define `DependsOn`, `LocalEvent`, event intersection, complement, and support extension. Prove probability is invariant under enlarging support.”
3. **Independence prompt.** “Prove that two local events with disjoint supports have multiplicative probabilities. Then prove the three-event version. Use only finite-sum factorization.”
4. **Geometry prompt.** “Define $\mathbb{Z}^d$ vertices as `Fin d -> Int`, define ℓ^1 balls, adjacency, bonds, and prove translation invariance and the ℓ^1 estimate used in subcritical decay.”
5. **Connectivity prompt.** “Define list paths, open paths, connectivity inside a finite set, exit events, and prove locality/increasingness of these events.”
6. **Russo prompt.** “Prove closed-pivotal Russo formula for finite local increasing events. You may first prove it for events on `Fin n -> Bool` and then transfer to finite supports.”
7. **Lemma 2 prompt.** “Assume the named shield-support lemma and pivotal-equivalence lemma; prove the differential inequality by sum decomposition and independence. After that, fill the two named lemmas.”

8. **Lemma 4 prompt.** “Prove the boundary inequality using the cluster $C_S(u)$, the last-exit lemma, a union bound, and the three-event independence theorem. Use the sharpened support of $\{C_S(u) = C_0\}$.”
9. **Main theorem prompt.** “Using Lemmas 2–5, prove $\tilde{p}_c = p_c$ and the final sharpness theorem. Do not change any theorem statement; if a supporting real-order lemma is missing, add it in `CriticalPoint.lean`.”

18 Final checklist before attempting the main theorem

Before closing `Main`, the following should already compile without `sorry`:

- C1.** finite Bernoulli probability is normalized and satisfies union bound;
- C2.** local event probability is invariant under support extension;
- C3.** disjoint-support independence works for two and three events;
- C4.** connection and exit events are local and increasing;
- C5.** $\varphi_p(S)$ is monotone in p ;
- C6.** Russo closed-pivotal formula is available for finite local increasing events;
- C7.** the random shield event $\{S = S\}$ is independent of internal S -edges;
- C8.** the pivotal equivalence used in Lemma 2 is proved for each oriented boundary edge;
- C9.** the generic ODE/log comparison lemma is proved;
- C10.** the boundary inequality is proved in the exact finite-volume form needed for $B = \Lambda_n^c$;
- C11.** the translation-to-origin estimate for $y \in \Lambda_L$ is proved;
- C12.** the real-order lemmas for $\tilde{p}_c$ and p_c handle endpoint cases.

If all checklist items compile, the final theorem should be a short assembly proof rather than a new mathematical argument.